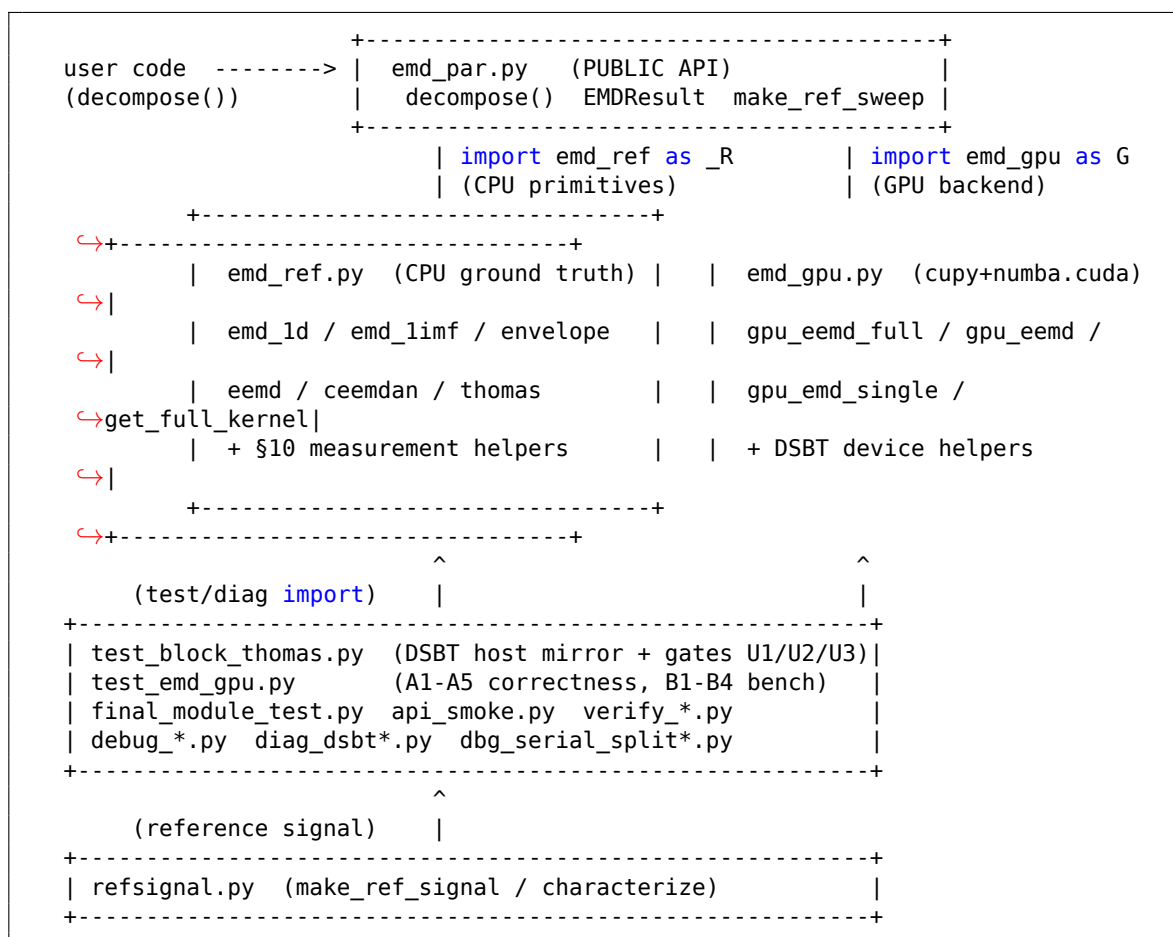


EMD / EEMD Parallel Suite — API Manual

Scope. This manual documents the full set of 1-D EMD / EEMD / CEEMDAN implementations in `emd_parallel/` and the **Decay-Seamed Block Thomas (DSBT)** spline solver that powers the GPU path. It is written for the production triplet (`emd_par.py`, `emd_ref.py`, `emd_gpu.py`) tested on the **RTX 4070** (8 GB) and **GB10** (128 GB unified) GPUs, and lists every related file that contains interacting code, with the exact function names that cross the module boundary.

Version note. The production files are the *un-suffixed* names (`emd_par.py`, `emd_ref.py`, `emd_gpu.py`). Sibling files such as `emd_par_v0.py`, `emd_par_v1.py`, `emd_par_v2.py`, `emd_par_v0_A2000.py`, `emd_ref_v0.py`, and `emd_gpu_v1.py` ... `emd_gpu_v5_dsbt.py` are **historical versions** kept for the review trail — do not import them; the current `emd_par.py` / `emd_gpu.py` already contain the DSBT + parallel-scan kernel.

1. Module map (who imports whom)



File	Role	Key interacting exports
'emd_par.py'	Public API — one entry point, CPU+GPU backends	'decompose', 'EMDResult', 'make_reference_sweep'
'emd_ref.py'	CPU ground truth (numpy) + \$!0 measurements	'emd_1d', 'emd_1imf', 'envelope', 'thomas', 'eemd', 'ceemdan', 'verify_thomas_vs_dense', 'load_imbalance_demo', 'mode_mixing_vs_E'

'emd_gpu.py'	GPU backend (cupy + numba.cuda), DSBT kernel	'gpu_eemd_full', 'gpu_eemd', 'gpu_eemd_single', 'get_full_kernel', '_pick_K'
'refsignal.py'	The 10-octave reference sweep (441 000 samples)	'make_ref_signal', 'characterize'
'test_block_thomas.py'	DSBT host-side reference + unit gates (no GPU)	'dsbt_solve', '_window_thomas_np', 'scan_extrema_2pass', 'build_system', 'rhs_of', 'decay_base', 'synth_knots', 'real_sweep_knots'
'dsbthomas.py' / 'dsbthomas_Gv0.py' / 'dsbthomas_Gv1.py'	Standalone DSBT solver prototypes + validation	'thomas_solve', 'dsbt_solve', 'generate_system', 'run_tests'
'test_emd_gpu.py'	Definitive GPU correctness + benchmark	(imports 'emd_ref', 'emd_gpu', 'emd_par', 'refsignal')
'final_module_test.py'	End-to-end module validation on the 441k sweep	(imports 'emd_par', 'emd_ref', 'refsignal', scipy 'hilbert')
'api_smoke.py'	Public-API smoke test (auto method, save/load, 1M/2M)	(imports 'emd_par')
'emd_green.py'	Green's-function decay-base analysis ($\rho = 2 - \sqrt{3}$)	'build_interior_system', 'thomas', 'green_subblock', 'report'
'dsbthomas.py' (decay section)	Empirical decay base + block-Thomas parallelism model	(script, no public API)
'debug_env.py', 'debug_kernel.py', 'debug_prod.py'	Throwaway single-sift GPU↔CPU diffing	(import 'emd_ref', 'emd_gpu')
'diag_dsbt.py' / 'diag_dsbt_v0.py' / 'diag_dsbt_v1_Gem.py'	DSBT seam-error profile diagnostics	(import 'emd_ref', 'test_block_thomas')
'dbg_serial_split.py' / 'dbg_serial_split_v0.py' / 'v0_Gem.py'	'clock64' phase-timing of the serial fraction	(host-instrumented kernel copy)
'verify_cpu_gpu_1.py' ... '_3.py', 'verify_full.py', 'validate.py', 'validate_final.py'	CPU↔GPU agreement / partition-of-unity checks	(import 'emd_ref', 'emd_gpu', 'emd_par')
'concurrency_test.py', 'concurrency_test2.py', 'conc3.py'	Concurrency / pool-accounting probes	(import 'emd_gpu')

'bench_tblock.py', 'non-stat_bench.py', 'final_scaling.py', 'scaling.py', 'hostgap.py', 'warm_timing.py', 'warm2.py', 'hilbert_sweep.py', 'capture_baseline_r3.py'	Benchmarks, scaling, host-overhead, baseline capture	(import 'emd_ref' / 'emd_gpu' / 'emd_par')
--	--	---

Rule of thumb. If you only want results, import `emd_par` and call `decompose()`. Everything else exists to (a) provide the CPU ground truth, (b) implement the GPU kernel, (c) prove the DSBT solver is correct, or (d) benchmark the two GPUs.

2. The parallelism model (read this first)

The design rests on one honest observation (stated in the `emd_ref.py` and `emd_par.py` docstrings):

- **One EMD of one signal is irreducibly sequential.** The siftings of a single signal, and the IMFs, are each strictly sequential. You cannot parallelize the sifting of one signal.
- **The ensemble axis is the parallel axis.** EEMD runs E independent EMDs of $x + \epsilon \text{std}(x) \text{noise}_e$ and averages the IMFs level-by-level. Trials are independent $\rightarrow$ embarrassingly parallel. $E = 1$ degenerates to plain EMD.
- **Within one sift**, the $O(N)$ work (extrema detect, spline eval, subtract, SD reduction) is embarrassingly parallel; the tridiagonal (2-dominant) spline solve is $O(\text{knots})$ and was the remaining serial bottleneck — now removed by **DSBT + a parallel extrema scan** in `emd_gpu.py`.

Two backends implement this:

- **CPU (`method='cpu'`)** — vectorized numpy + `ProcessPoolExecutor` with **dynamic** scheduling (`as_completed`), so a slow trial never idles the pool (load-imbalance handling). This is the robust workhorse for 100k–2M samples.
- **GPU (`method='gpu'`)** — cupy + numba.cuda, **one thread block per trial**, the whole IMF loop inside a single kernel, $O(N)$ spline evaluation via binary-search interval lookup, and **trial batching** to fit device memory.
- **`method='auto'`** — GPU if a device is available and the working set fits, else CPU.

2.1 The 2-dominant spline system (shared math)

Both backends solve the same natural-BC cubic-spline system for the second derivatives M of the envelope through the extrema knots. With knot spacings $H[j] = \text{tm}[j+1] - \text{tm}[j]$ and $n = p - 2$ interior unknowns:

$$\begin{aligned}
 d[m] &= 2(H[m] + H[m+1]) && \text{(diagonal)} \\
 l[m] &= H[m] && \text{(sub-diagonal, } m \geq 1) \\
 u[m] &= H[m+1] && \text{(super-diagonal, } m \leq n-2) \\
 r[m] &= 6 \left(\frac{ym[m+2] - ym[m+1]}{H[m+1]} - \frac{ym[m+1] - ym[m]}{H[m]} \right) && \text{(RHS)}
 \end{aligned}$$

$d = 2(l + u)$ exactly, so the matrix is **factor-2 diagonally dominant** — Thomas' algorithm needs **no pivot** and is backward-stable for a single step. This is built identically in `emd_ref._build_interior_system/_rhs`, `test_block_thomas.build_system/rhs_of`, and inside the GPU kernel.

3. emd_par.py — the public API

```
import emd_par as M
result = M.decompose(x, method='auto', E=64, tau=0.25)
result.imfs          # ndarray [n_modes, N] (ensemble-averaged IMFs)
result.residual      # ndarray [N]
result.n_modes       # int
result.recon_error() # max|x - (sum(imfs) + residual)| (partition of unity)
result.save(path)    # compressed .npz
M.EMDResult.load(path)
```

3.1 decompose(x, ...)

```
decompose(x, method='auto', E=64, eps=0.2, tau=0.25, max_sifts=50,
          max_imf=-1, min_extrema=4, end='natural', n_workers=None,
          seed=0, parallel=True, batch=None, nthreads=128,
          mem_limit_gb=None) -> EMDResult
```

Param	Default	Meaning
'x'	—	1-D array. Must be 1-D and have length ≥ 8 (else 'ValueError').
'method'	"auto"	"auto" / "cpu" / "gpu".
'E'	'64'	Ensemble size. $E = 1$ = plain EMD, $E > 1$ = EEMD.
'eps'	'0.2'	Noise amplitude as a fraction of $\text{std}(x)$ (EEMD only).
'tau'	'0.25'	User stop-criterion — SD tolerance (Rilling-Flandrin-Gonçalves). Lower 'tau' = more siftings = closer to a "true" IMF, slower.
'max_sifts'	'50'	Hard cap on siftings per IMF (safety).
'max_imf'	'-1'	Stop after this many IMFs. Semantics differ by backend (see below).

'min_extrema'	'4'	Stop decomposing when the residual has fewer than this many extrema. CPU only — the GPU kernel hardcodes '4'.
'end'	"natural"	Spline end handling: "natural" or "mirror" (Rilling et al.). CPU only — the GPU kernel is natural-BC only.
'n_workers'	'None'	CPU process-pool size ('None' = auto: 'max(1, min((E-1)//2, cpu_count))').
'seed'	'0'	RNG seed (reproducibility).
'parallel'	'True'	CPU: 'True' = 'ProcessPoolExecutor' (dynamic), 'False' = serial (ground-truth baseline). Ignored by GPU.
'batch'	'None'	GPU: max trials resident at once ('None' = auto from free memory).
'nthreads'	'128'	GPU threads per block.
'mem_limit_gb'	'None'	GPU: explicit device-memory ceiling in GB for auto-batching ('None' = measure driver-free + cupy-pool cached memory).

Returns an EMDResult. stats keys:

- common: method, E, n_modes, x (the input, retained so recon_error() can check the partition of unity).
- CPU: total_sifts ($E = 1$) or per_trial_total_sifts ($E > 1$), n_workers.
- GPU: batch, per_trial_total_sifts, gpu_time_s, max_imf_reached.

max_imf semantics (important):

- **CPU:** -1 = decompose fully (no cap).
- **GPU:** -1 = **hard cap 64** (memory-bound: per-trial buffers would not fit beyond this on an 8 GB card). If the cap binds, a RuntimeWarning is emitted and stats['max_imf_reached'] is True — the residual may still contain oscillatory content.

3.2 EMDResult (dataclass)

Member	Type	Notes
'imfs'	'ndarray [n_modes, N]'	Ensemble-averaged IMFs.
'residual'	'ndarray [N]'	$x - \sum \text{imfs}$.
'n_modes'	'int'	Number of IMFs.
'stats'	'dict'	See 3.1.

'recon_error()'	'float'	$\max x - (\sum \text{imfs} + \text{residual}) $. Returns '0.0' if 'stats' has no 'x' (the identity holds by construction).
'save(path)'	—	'np.savez_compressed'; stores 'imfs', 'residual', 'n_modes', and scalar stats.
'EMDResult.load(path)'	'EMDResult'	Static loader; 'allow_pickle=False'.

3.3 make_reference_sweep(...)

```
make_reference_sweep(FS=44100, T=10.0, F1=20.0, F2=20000.0,
                    noise_level=0.01, seed=0) -> (x, t)
```

10-octave log sine sweep (20 Hz → 20 kHz) over 10 s at $F_s = 44.1$ kHz ($N = 441\,000$), plus white noise at noise_level (fraction of peak). This is the emd_par-side twin of refsignal.make_ref_signal (same formula); use either. refsignal.py is the one the test suite imports.

3.4 Internal backends (interacting code)

- `_cpu_decompose(...)` — EMD ($E = 1$) or EEMD ($E > 1$) on CPU. Calls `emd_ref.emd_1d` (via `import emd_ref as R`). EEMD uses **streaming level-by-level accumulation** (memory $O(n_{\text{modes}} \cdot N)$, independent of E) and **per-trial SeedSequence children** (`np.random.SeedSequence(seed).spawn(E)`), so the full signal x is pickled once per pool (via the `_pool_init` initializer into the child-global `_SHARED`) rather than into each of the E tasks. `_emd_trial` is a **pure** function (no globals) so serial and concurrent calls are thread-safe.
- `_gpu_decompose(...)` — EEMD on GPU. Caps `max_imf` at 64 when -1, computes the auto batch from device memory, generates per-batch noise, and calls `emd_gpu.gpu_eemd_full` (via `import emd_gpu as G`). Averages per mode over the trials that produced that mode.
- `_gpu_available()` — probes `cupy.getDeviceCount()`.
- `_accumulate(avg, cnt, imfs, N)` — grows the running average/cnt as trials finish (dynamic scheduling).

GPU-path caveats. `emd_par._gpu_decompose` accepts `end` and `min_extrema` but the kernel is **natural-BC only** with `min_extrema=4` hardcoded, so those two parameters are **silently ignored on the GPU path**. Pass `end='mirror'` or a custom `min_extrema` and use `method='cpu'`.

4. emd_ref.py — CPU reference (ground truth)

Ground-truth CPU implementations plus the \$10 supporting measurements. Imported by `emd_par` (as `_R`) and by most test/diag scripts (as `R`).

4.1 Spline envelope

```
envelope(t, x, want, end='natural') -> (env, n_knots)
```

Cubic-spline envelope of x through its local **maxima** (`want='max'`) or **minima** (`want='min'`), evaluated at all samples t .

- end='natural' — natural BC ($M_1 = M_p = 0$) on the extrema knots.
- end='mirror' — Rilling et al.: reflect the 2nd/penultimate extremum past each edge to form extra knots, fit, evaluate on the **original** grid.

Returns the envelope array and the knot count. Backed by the internal `_build_interior_system`, `_rhs`, `thomas`, `_spline_eval`.

4.2 Single-signal EMD (the atomic unit)

```
emd_ld(x, t=None, tau=0.25, max_sifts=50, max_imf=-1,
       min_extrema=4, end='natural') -> (imfs, residual, stats)
```

Full EMD of a 1-D signal. `imfs` is a **list** of ndarray (convert with `np.array`). `stats = dict(n_modes, total_sifts, per_mode_sifts)`. `max_imf=-1` = no cap (CPU). Stops when the residual has fewer than `min_extrema` extrema or hits `max_imf`.

```
emd_limf(x, t=None, tau=0.25, max_sifts=50, end='natural') -> (imf, n_sift)
```

Sift to the **first IMF only** (used by CEEMDAN).

4.3 Ensemble (the parallel axis)

```
eemd(x, E=100, eps=0.2, parallel=True, n_workers=None, seed=0,
     **kw) -> (avg, resid, res)
```

E independent EMDs of $x + \epsilon \text{std}(x)$ noise _{e} , IMFs averaged level-by-level (trials may yield different `n_modes`). `res` is the list of per-trial (`imfs`, `resid`, `stats`, `time`). `**kw` passes through `tau`, `max_sifts`, `min_extrema`, `end`.

```
ceemdan(x, E=100, eps=0.2, parallel=True, n_workers=None, seed=0,
       **kw) -> (c_imfs, resid, dict(E, n_imfs))
```

CEEMDAN (Torres 2011 / Colominas 2014 improved). Staircase: **sequential in IMF index, parallel over the E trials per stage**. Each noise IMF is normalized by its 1st-IMF std (the Colominas improvement).

Supporting trial workers (top-level, picklable): `_trial_eemd`, `_trial_limf`, `_run_trials(parallel, ↪n_workers, fn, tasks)` (the dynamic-scheduler wrapper), `_gen_noise(E, N, seed)`.

4.4 §10 supporting measurements

```
verify_thomas_vs_dense(d, l, u, r) -> float
```

Cross-checks the no-pivot thomas solve against an **independent dense** `np.linalg.solve` of the identical system; returns $\max |\Delta|$ ($\sim$ machine epsilon). The GPU/DSBT solve is compared against this same ground truth.

```
load_imbalance_demo(x, E=32, n_workers=4, seed=0) -> dict
```

Runs trials serially to record (work, time) per trial, then computes static-round-robin vs dynamic lower-bound makespan. Returns `works`, `times`, `n_sift_min/max/mean`, ↪`static_makespan`, `dynamic_lb`, `serial`, `imbalance_ratio`.

```
mode_mixing_vs_E(x, Es=(1, 4, 16, 64, 256), seed=0, **kw) -> dict
```

Two close tones with different AM → mode mixing. Returns the EEMD IMFs per E (a proxy for the $O(E^{-1/2})$ Monte-Carlo rate).

```
thomas(d, l, u, r) -> ndarray    # 0(n) Thomas, no pivot (2-dominant)
```

5. emd_gpu.py — GPU backend (cupy + numba.cuda)

One thread block per trial; the full IMF loop lives inside a single @cuda.jit kernel. Spline evaluation is $O(N)$ (binary-search interval lookup). The tridiagonal solve is **DSBT** (seam-window) plus a **parallel extrema scan** — the two changes that removed the single-thread serial fraction (see §7 and BLOCK_THOMAS_PLAN.md).

5.1 Public entry points

```
gpu_eemd_full(x, E=64, tau=0.25, max_sifts=50, max_imf=12, seed=0,
              K=None, nthreads=128, batch=None, B=250, S=30,
              EXACT=False) -> (imfs, nsift, nmode)
```

Full-IMF GPU EEMD over a **batch** of E trials. x is **already perturbed**, shape (E, N) (for $E = 1$ pass a 1-D array). Returns imfs ndarray of shape (E, maxm, N) **trimmed to** `max(nmode)` across the batch, plus per-trial nsift (int32) and nmode (int32).

Vestigial params. seed and batch are in the signature but **not used** here — gpu_eemd_full launches all E blocks at once. Real noise generation and trial batching live in emd_par._gpu_decompose, which calls this on each batch slice. A 1-D input with $E > 1$ raises ValueError (regression guard for the silent- $E=1$ bug).

```
gpu_eemd(x, E=64, eps=0.2, tau=0.25, max_sifts=50, seed=0,
         K=None, nthreads=128, B=250, S=30, EXACT=False)
         -> (mean_imf, nsift, imfs)
```

First-IMF-only EEMD: each trial sifts only IMF 0 (MAXIMF=1), noise is generated here from seed ($\epsilon \text{std}(x)$). Returns the ensemble-averaged first IMF $(N,)$, per-trial nsift, and the per-trial IMFs (E, N) .

```
gpu_emd_single(x, t=None, tau=0.25, max_sifts=50, K=None, nthreads=128,
               B=250, S=30, EXACT=False) -> (imf, n_sift)
```

Single-trial full EMD of one 1-D signal ($E = 1$). Returns the first IMF $(N,)$ and its sift count. Used for per-sift GPU↔CPU diffing (debug_prod.py).

```
get_full_kernel(N, K, MAXIMF, nthreads, B=250, S=30, EXACT=False) -> kernel
```

Returns a **cached** compiled kernel (cache key = $(N, K, \text{MAXIMF}, \text{nthreads}, B, S, \text{EXACT})$). Callers normally use the three entry points above, which call this.

```
_pick_K(N) -> int    # max(16, N//2) (knot-index capacity)
```

5.2 Device helpers (@cuda.jit(device=True, inline=True))

- `_thomas(d, l, u, r, x, c_, n)` — exact serial Thomas (used when EXACT=True, for in-kernel A/B debugging; **not** the production path).
- `_window_thomas(dd, ll, uu, rr, Mf, buf, w0, w1, o0, o1, B2S)` — **DSBT**: Thomas over the window $[w_0 \dots w_1]$ with the **edge couplings dropped** (zero BC at the window rims). buf is a 1-D private per-thread slice of size $2(B+2S)$. Only the **owned** $[o_0 \dots o_1]$ values are written back to Mf (halo values are discarded — ownership is disjoint, so no halo element is written twice).

- `_scan_extrema(arr, kt, ky, A, Bm, N, K, tid, nthreads, want_max, collect)` — **tile-based parallel extrema detect + compact**, preserving time order. Two passes: strided detect → per-tile count → (when collect) compact into kt/ky. This is what removed the 78.8% serial scan fraction.
- `_spline_eval_bin(tm, ym, M, t, p)` — cubic-spline evaluation at sample t via **binary search** over the knot times (the $O(\log p)$ interval lookup that replaced the old linear scan).

5.3 Kernel factory & buffers

`_make_full_kernel(N, K, MAXIMF, nthreads, B, S, EXACT)` builds the kernel. Per-trial device buffers allocated in the entry points:

Buffer	Shape	Purpose
'S_arr'	'(E, N)'	input (perturbed) signal
'resid' / 'h' / 'eu'	'(E, N)' each	residual, current sift h, upper envelope
'out'	'(E, max_imf, N)'	IMF output
'W'	$(E, 9K)$	9 slabs of K : 'ky, H, dd, ll, uu, rr, xx, cc, Mf'
'W_int'	'(E, K)' int32	'kt' (knot sample indices)
'Wwin'	$(E, n_{\text{threads}}, 2(B + 2S))$	private per-thread DSBT window buffer
'nsift' / 'nmode'	'(E,)' int32	per-trial counters

Per-trial footprint. Two (consistent, slightly different) models are in play — use the one that matches your question:

- `test_emd_gpu.footprint_bytes(N, max_imf) =  $N \cdot 8 \cdot (4 + \text{max\_imf}) + 10 \cdot (N/2) \cdot 8$` — counts S, resid, h, eu ($4N$) + out ($\text{max_imf} \cdot N$) + w (10 slabs of $N/2$). This is the number in the benchmark tables: **≈ 84 MB/trial** at $N = 441\,000$, $\text{max_imf} = 16$.
- `emd_par._gpu_decompose's auto-batch model per_trial_gb =  $(1 + \text{max\_imf}) \cdot N \cdot 8 + 9 \cdot (N/2) \cdot 8 + (N/2) \cdot 4$` (the extra $(N/2) \cdot 4$ is the int32 `w_int` slab) — **≈ 73 MB/trial** at the same config. It under-counts the $4N$ signal slabs on purpose so auto-batching stays conservative against the driver.

$B = 250$ (owned block), $S = 30$ (halo), $n_{\text{threads}} = 128$ are the defaults (see §7 for the error budget that fixes S).

6. refsignal.py — reference signal

```
make_ref_signal(noise_level=0.01, seed=0) -> (x, t)
characterize(x, t) -> dict
```

The user-specified 10-octave log sweep: 20 Hz → 20 kHz, 10 s, $F_s = 44.1$ kHz → $N = 441\,000$, + white noise at 1/100 of peak. Log-sweep phase $\phi(t) = 2\pi f_1 T ((f_2/f_1)^{t/T} - 1)/\ln(f_2/f_1)$. `characterize` returns N, nmax, nmin, n_extrema, total_cycles, spc_lo, ↪ spc_hi, peak, std. This is the canonical test signal for `test_emd_gpu.py`, `final_module_test.py`, and `test_block_thomas.real_sweep_knots`.

7. Decay-Seamed Block Thomas (DSBT) — the algorithm

Problem. The exact Thomas solve is $O(n)$ but strictly sequential — one CUDA thread walked $\sim 441k$ interior unknowns per envelope, which (per the clock64 Phase-0 measurement in BLOCK_THOMAS_PLAN.md §3.1) was the serial fraction that capped GPU speed. The extrema **scans** were actually the bigger culprit (78.8% of τ_{block}); DSBT + the parallel scan together remove $\sim$ all of it.

DSBT (Option A, the production solver). Partition the n_{int} interior unknowns into contiguous **owned blocks** of $B = 250$. Each block solves its **window** = owned region $\pm S = 30$ **halo** ($B + 2S = 310$ unknowns) as a *standalone* tridiagonal system — the external couplings at the two window edges are **dropped** (equivalently, zero BC at the window rims). The **owned** values are the answer; the halo values are discarded (each halo element is solved redundantly by its neighbour, never written twice — ownership is disjoint). Blocks are assigned round-robin over the 128 threads (`for blk in range(tid,  $\hookrightarrow$ nb, nthreads)`), so the solve is parallel.

Why it's accurate (the error budget). The Green's function of the 2-dominant spline system decays as $\rho^{|i-j|}$ with local base $r_i \leq 2 - \sqrt{3} = 0.267949$ for **all** spacings (AM-GM: $r_i + 1/r_i = 2(1 + H_i/H_{i+1}) \geq 4$). Uniform spacing is the *slowest* decay; measured bases: uniform 0.267949, $\text{CV} \approx 0.4 \rightarrow 0.2502$, $\text{CV} \approx 0.8 \rightarrow 0.2269$, $\text{CV} \approx 1.5 \rightarrow 0.2136$, alternating 1/10 spacing $\rightarrow 0.1616$. An owned point is $\geq S = 30$ from both window rims, so the seam error is $O(\rho^{30} \cdot \text{scale}) \leq 6.9 \times 10^{-18} \cdot C \cdot \text{scale}$ — **10 $\times$ below double-precision** $\varepsilon \approx 2.2 \times 10^{-16}$, and ~ 4 orders below the existing GPU-vs-CPU noise floor ($1.4\text{e-}13$). $S = 20$ would give 3.6×10^{-12} (inside the gate but no margin), so $S = 30$ is kept.

The parallel extrema scan (companion change). The three scan sites (residual count per IMF; MAX and MIN knot collect per sift) reduce to *detect 1-stencil extrema, count/compact in time order*. Two passes over the block's 128 threads (no extra global memory):

1. **Pass 1 (count):** strided detect $\rightarrow$ local count in shared; Hillis-Steele inclusive scan over the 128 counts (7 synctreads); total is exact.
2. **Pass 2 (compact):** re-detect in the same strided order, writing `kt[off + local] = i`, `ky[off + local] = hv[i]` (capped at K).

Writes happen in strictly ascending i , so kt/ky are **bit-identical** to the old single-thread scan. The $K = N/2$ cap can never fire.

7.1 DSBT reference & verification files (interacting code)

`test_block_thomas.py` — the **host-side numpy mirror** of the device algorithm (no GPU, no kernel). This is the “mirror of the device algorithm exactly” the plan requires; if the mirror passes, the device has the same math modulo float64 associativity. Public functions:

Function	Mirrors	Role
<code>'dsbt_solve(d, l, u, r, B=250, S=30)'</code>	device <code>'_window_thomas'</code> loop	full DSBT solve $\rightarrow$ interior 'x'
<code>'_window_thomas_np(d, l, u, r, x, c_, w0, w1)'</code>	device <code>'_window_thomas'</code>	seam-window Thomas, edge couplings dropped
<code>'scan_extrema_2pass(arr, want_max, nthreads=128)'</code>	device <code>'_scan_extrema'</code>	host emulation of the two-pass scan
<code>'scan_extrema_serial(arr, want_max)'</code>	(serial reference)	ground truth for U2
<code>'build_system(tm)'</code> / <code>'rhs_of(tm, ym, H, n)'</code>	<code>'emd_ref_build_interior_system/ref_rhs'</code>	build the 2-dominant system
<code>'decay_base(d, l, u, n)'</code>	—	measure the Green's-function decay base 'r'

'_dense(d, l, u, n)'	—	dense matrix for the independent solve
'synth_knots(kind, p=5000, seed=1)'	—	'uniform' / 'cv04' / 'alt110' / 'cv15' knot sets
'real_sweep_knots(n_knots=5000, seed=0)'		real knots from the reference-sweep EMD (imports 'refsignal' + 'emd_ref.envelope')

Unit gates (main(), exit 0 = all pass):

- **U1 (solver vs exact):** for real-sweep, uniform, CV0.4, alternating-1/10, CV1.5 knot sets — $\max |\Delta M| / \max |M| \leq 10^{-15}$ vs emd_ref.thomas (exact) and vs np.linalg.solve (independent). Expected $\sim 1e-17$.
- **U2 (scan equivalence):** the two-pass scan is **bit-identical** to the serial scan on random / flat-run / plateau / single-extremum / constant / monotone / alternating cases, max and min.
- **U3 (decay base):** measured base $\leq 2 - \sqrt{3}$ on all U1 knot sets.

dsbthomas.py / dsbthomas_Gv0.py / dsbthomas_Gv1.py — standalone DSBT prototypes + validation (no EMD, no GPU): thomas_solve (exact), dsbt_solve(d, l, u, r, b, s) (seam-window), generate_system(N_knots, CV) (build a 2-dominant system with a given knot-spacing CV), run_tests() (error-vs-seam-width curve, error localization, non-uniformity CV stress). dsbthomas.py (the decay section) additionally prints the empirical decay base on a non-uniform knot set and the block-Thomas parallelism model at $N = 441\,000$.

diag_dsb.py / diag_dsb_v0.py / diag_dsb_v1_Gem.py — diagnose the DSBT **seam-error profile** (where the max error lands relative to the block boundaries) on uniform / cv04 / cv15 knot sets. Import emd_ref and test_block_thomas.

emd_green.py — the *theoretical* side: builds the interior system, inverts a dense sub-block to extract the Green's function, fits the decay base ρ , and compares to theory $\rho = 2 - \sqrt{3}$ and $C = 1/(2\sqrt{3})$ on a regular 50 Hz sinusoid and an irregular chirp+AM signal. Confirms the error budget that justifies $S = 30$.

8. Testing & benchmarking (how it's verified)

File	What it does	Imports
------	--------------	---------

'test_emd_gpu.py'	Definitive GPU test + benchmark. Part A (hard assertions): A1 per-sift GPU vs CPU $\leq 10^{-13}$; A2 full-IMF per-IMF $\leq 10^{-8}$ + $ \Delta n_{\text{mode}} \leq 1$ + CPU partition-of-unity $\leq 10^{-10}$; A3 determinism (same seed bit-identical); A4 different seed differs; A5 1-D + $E > 1$ raises. Part B (report): B1 'T_block', B2 saturation-concurrency sweep, B3 device footprint, B4 host overhead.	'emd_ref', 'emd_gpu', 'emd_par', 'refsignal'
'test_block_thomas.py'	DSBT host mirror + gates U1/U2/U3 (§7.1).	'emd_ref', 'refsignal'
'final_module_test.py'	End-to-end on the 441k sweep: 'decompose()' for CPU E=1/8/64 + GPU E=64, partition of unity, streaming correctness, load-imbalance, mode-mixing reduction (IMF-1 Hilbert IF spread vs E).	'emd_par', 'emd_ref', 'refsignal', scipy 'hilbert'
'api_smoke.py'	Public-API smoke: 'decompose()' default (auto), save/load round-trip, 1M and 2M-sample signals.	'emd_par'
'verify_cpu_gpu_1/2/3.py', 'verify_full.py', 'validate.py', 'validate_final.py'	CPU↔GPU per-IMF agreement, partition of unity, full-decomposition checks.	'emd_ref', 'emd_gpu', 'emd_par'
'concurrency_test.py', 'concurrency_test2.py', 'conc3.py'	Concurrency and cupy pool-accounting probes.	'emd_gpu'

'bench_tblock.py', 'final_scaling.py', 'scaling.py', 'non-stat_bench.py', 'host-gap.py', 'warm_timing.py', 'warm2.py', 'hilbert_sweep.py'	'T_block', scaling, non-stationary signals, host-overhead, warm timing, Hilbert-IF sweep.	'emd_ref' / 'emd_gpu' / 'emd_par'
'capture_baseline_r3.py'	Captures the pre-change GPU baseline → 'baseline_r3.npz' (the Phase-3 bit-exact reference).	'emd_gpu'
'debug_env.py', 'debug_kernel.py', 'debug_prod.py'	'de-Throwaway single-sift GPU↔CPU envelope / h1 diffing.	'emd_ref', 'emd_gpu'
'dbg_serial_split.py' / '_v0.py' / '_v0_Gem.py'	'clock64'-instrumented kernel copy that produced the serial/parallel phase split.	(host-instrumented)

Run the gates (from emd_parallel/, in the venv):

```
.venv/Scripts/python.exe test_block_thomas.py    # DSBT host gates (no GPU)
.venv/Scripts/python.exe test_emd_gpu.py          # GPU correctness + bench
.venv/Scripts/python.exe final_module_test.py     # end-to-end module
.venv/Scripts/python.exe api_smoke.py             # public-API smoke
```

9. Measured performance (RTX 4070 vs GB10)

From Benchmark_Thomas_vs_Decay-Seamed_Block-Thomas.md (reference sweep $N = 441\,000$, $\max_{\text{imf}} = 16$, warm medians). Part A correctness is identical across all four configs (A1 $4.4\text{e-}16$, A2 $1.4\text{e-}13$ per-IMF, A3 bit-identical, A5 raises).

Per-block time τ_{block} ($E = 1$, the single-trial scalar latency):

	RTX 4070 (Thomas)	RTX 4070 (DSBT)	GB10 (Thomas)	GB10 (DSBT)
' $T_{\text{block}}@N = 441\,000$ '	4.61 s	0.50 s	13.67 s	1.42 s
' $T_{\text{block}}@N = 110\,250$ '	1.11 s	0.12 s	3.33 s	0.34 s
speedup	—	** $\sim 9 \times$ **	—	** $\sim 10 \times$ **

Saturation sweep @ $N = 441\,000$ ($C_{\text{est}} = E \cdot T_{\text{block}}/\text{wall}$):

E	4070 Thomas	4070 DSBT	GB10 Thomas	GB10 DSBT
8	5.97 s	0.83 s	16.43 s	1.73 s
32	11.26 s	1.77 s	29.57 s	3.87 s
64	12.05 s	2.54 s	30.23 s	4.57 s
128	21.75 s	11.08 s	36.46 s	6.22 s

Readings.

- DSBT + parallel scan cut the single-trial serial fraction $\sim 9\text{--}10\times$ on both chips, confirming the Phase-0 Amdahl model (serial was 95.8% / 95.5%).
- The **new floor** is the already-parallel spline-eval + SD phase. On the GB10 it is ~ 0.46 s (1:64 FP64 segmentation, 0.132 TFLOPS measured) — the GB10's 128 GB pool is what lets it win at large E (no VRAM paging).

- At $E = 128$, $N = 441\,000$ the 4070 hits ~ 10.5 GB (VRAM paging / OOM territory on an 8 GB card) — that is why `emd_par` **batches** and why the GPU `max_imf` is capped at 64. The GB10 runs the same E in unified memory with no paging.
- Host overhead of `emd_par` over the raw kernel is small: ~ 0.11 – 0.12 s on the 4070, ~ 0.03 – 0.08 s on the GB10 (B4).

10. Quick recipes

```
import emd_par as M, numpy as np

x, t = M.make_reference_sweep()           # 441 000-sample 10-octave sweep

# 1) One-liner, best available backend
r = M.decompose(x, tau=0.25)              # E=64 EEMD, auto method
print(r.n_modes, r.recon_error())

# 2) Plain EMD on CPU (ground truth, no noise)
r = M.decompose(x, method='cpu', E=1, tau=0.25)

# 3) EEMD on GPU, control memory
r = M.decompose(x, method='gpu', E=128, tau=0.25,
                max_imf=16, batch=16, mem_limit_gb=7.0)

# 4) Mirror end handling (CPU only)
r = M.decompose(x, method='cpu', E=8, end='mirror', min_extrema=6)

# 5) Save / load
r.save('sweep.npz'); r2 = M.EMDResult.load('sweep.npz')

# 6) Raw GPU kernel (advanced; already-noised batch)
import emd_gpu as G
imfs, nsift, nmode = G.gpu_eemd_full(x[None, :], E=1, tau=0.25,
                                     max_sifts=50, max_imf=16)

# 7) CPU ground truth / ensemble directly
import emd_ref as R
imfs, resid, st = R.emd_ld(x, tau=0.25)      # plain EMD
avg, resid, res = R.eemd(x, E=100, eps=0.2)  # EEMD
c, resid, meta = R.ceemdan(x, E=100)         # CEEMDAN
```

11. Pitfalls & caveats (read before relying on a result)

1. **max_imf differs by backend.** `-1` = full (CPU) but = **cap 64** (GPU). On the GPU a bound cap raises `RuntimeWarning` and sets stats['`max_imf_reached`'].
2. **end and min_extrema are CPU-only.** The GPU kernel is natural-BC only with `min_extrema=4` hardcoded; those two `decompose()` args are silently ignored when `method='gpu'`.
3. **GPU vs CPU is chaotic.** Per-IMF agreement is $\sim 1e-8$ (A2), per-sift $\sim 1e-13$ (A1); a full decomposition amplifies 1-ulp differences by $\times 10^2$ – 10^3 , so exact bit-equality between backends is **not** expected (only same-backend same-seed is bit-identical, A3).
4. **gpu_eemd_full trims its output** to `max(nmode)` across the batch — different seeds give different depths; compare over the common depth.
5. **gpu_eemd_full's seed/batch are vestigial.** Noise + batching happen in `emd_par._gpu_decompose`. Call `decompose(method='gpu')` for the real EEMD path; use `gpu_eemd_full` directly only with an already-noised batch.

6. **cupy pool accounting.** `memGetInfo()` alone undercounts free memory after large allocations (the pool keeps its high-water mark), which would silently degrade auto-batching to 1. `emd_par._gpu_decompose` reads the pool's cached bytes too; the raw-kernel benchmarks call `free_all_blocks()` for clean timings.
7. **Windows NVVM bootstrap.** `emd_gpu._bootstrap_nvvm()` preloads `cuda.dll/nvvm.dll` from `./cuda_libs/` or `CUDA_HOME` on Windows — keep those DLLs reachable or numba's CUDA JIT will fail to link.
8. **Don't import the versioned files** (`emd_par_v0/v1/v2`, `emd_gpu_v1..v5`, `emd_ref_v0`, `*_A2000`). They are review-trail snapshots; the un-suffixed `emd_par.py` / `emd_gpu.py` already ship DSBT + parallel scan.